Local Search

EECS 492

Introduction to Artificial Intelligence

Reminders

- This week's discussion sections: local search and adversarial search
- HW3 to be released tomorrow
- Grades for attendance and ethics to be updated soon

Today's Class

- Local Search
  - Hill Climbing
  - Simulated Annealing
  - Beam Search
  - Genetic Algorithms

Local Search

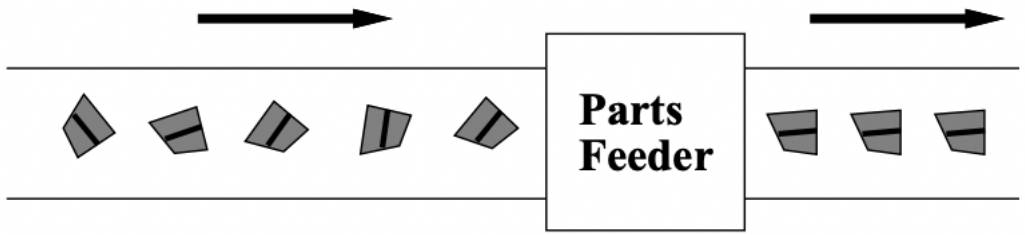
Task Environments

- An agent operates in a task environment
  - Some environments are harder than others!

- We have different properties to describe the kind of environment than an agent operates in

Fully vs. Partially Observable

- Fully observable: Agent's sensors give it access to the complete state of the environment (or all relevant aspects)
  - No internal state needed!
  - Example: chess
- Partially observable:
  - Caused by noisy, inaccurate, or missing sensors
  - Example: poker



- Unobservable: no sensors
  - Example: in manufacturing, there are methods that take in a part (in an unknown initial position) and orient it correctly without any sensors

Image: https://goldberg.berkeley.edu/pubs/algo93.pdf

Deterministic vs. Nondeterministic

- Deterministic: state of environment is completely determined by the current state of the environment and the action executed by the agent
  - Example: image analysis

- Nondeterministic / stochastic: otherwise
  - Example: poker

Known vs. Unknown

- Unknown: Agent does not know the laws of the environment
  - Agent has to learn how the world works in order to make good choices

- Known:
  - Outcome (or outcome probabilities if the environment is stochastic) for all actions are given

Search So Far…

- We looked at one category:
  - Observable
  - Deterministic
  - Known environments
  - Solution: sequence of actions

  - Systematic exploration of the space
  - Keep 1+ paths in memory and record alternatives
  - When goal is found – path is the solution

  - Do we really care about the path??

Local Search

- Local search
  - Observable
  - Deterministic
  - Known environments
  - ~~Solution: sequence of actions~~

Search So Far

Uninformed Search

- No extra information
- Care about path to solution

Breadth-first search
Uniform-cost search
Depth-first search
(and variations – limited DFS, iterative deepening DFS)

Informed Search

- Extra information (heuristic)
- Care about path to solution

Greedy best-first search
A*

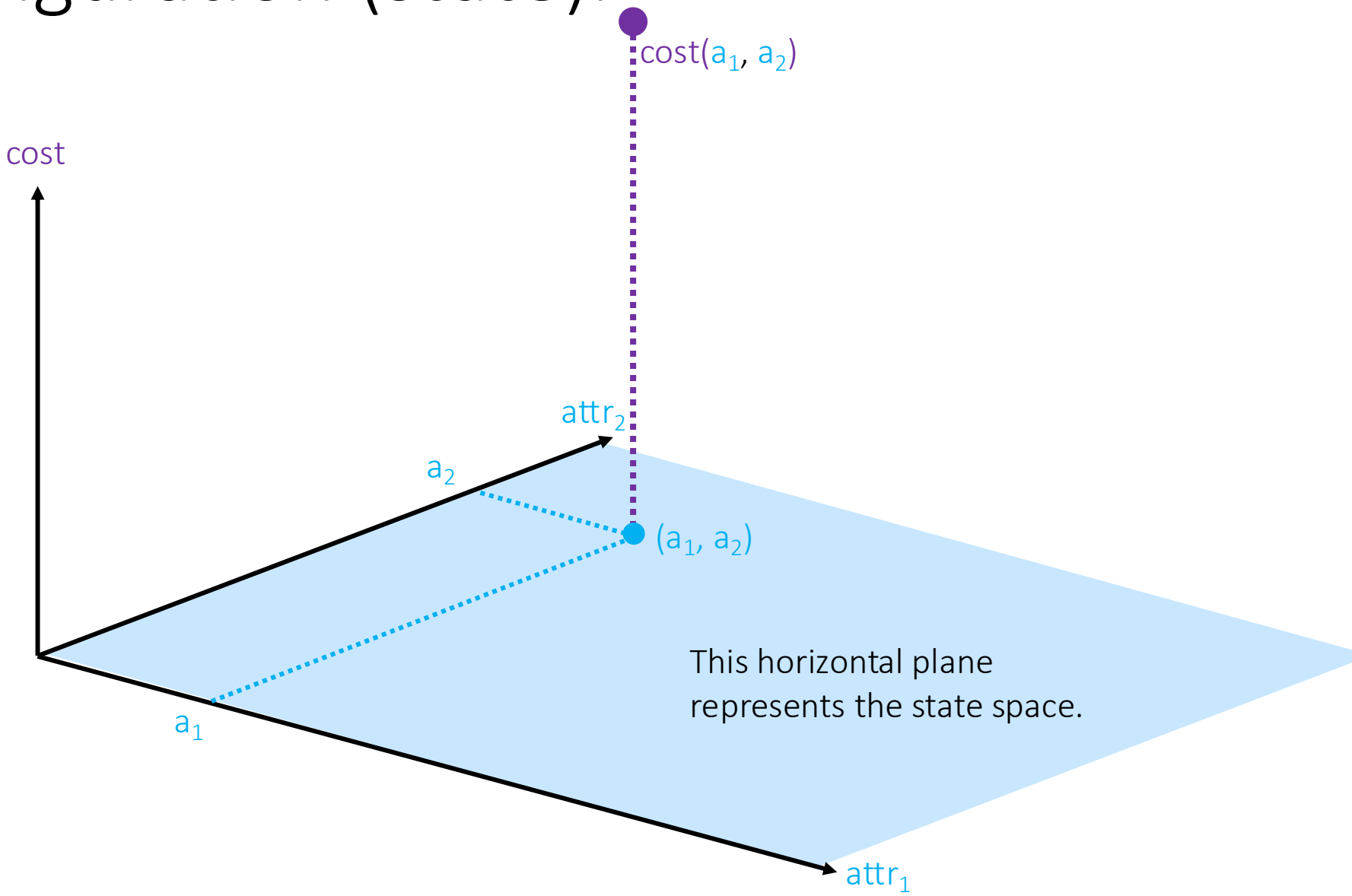
Local Search

- Don’t care about path to solution

Local search: finding the best configuration (state).

Suppose:

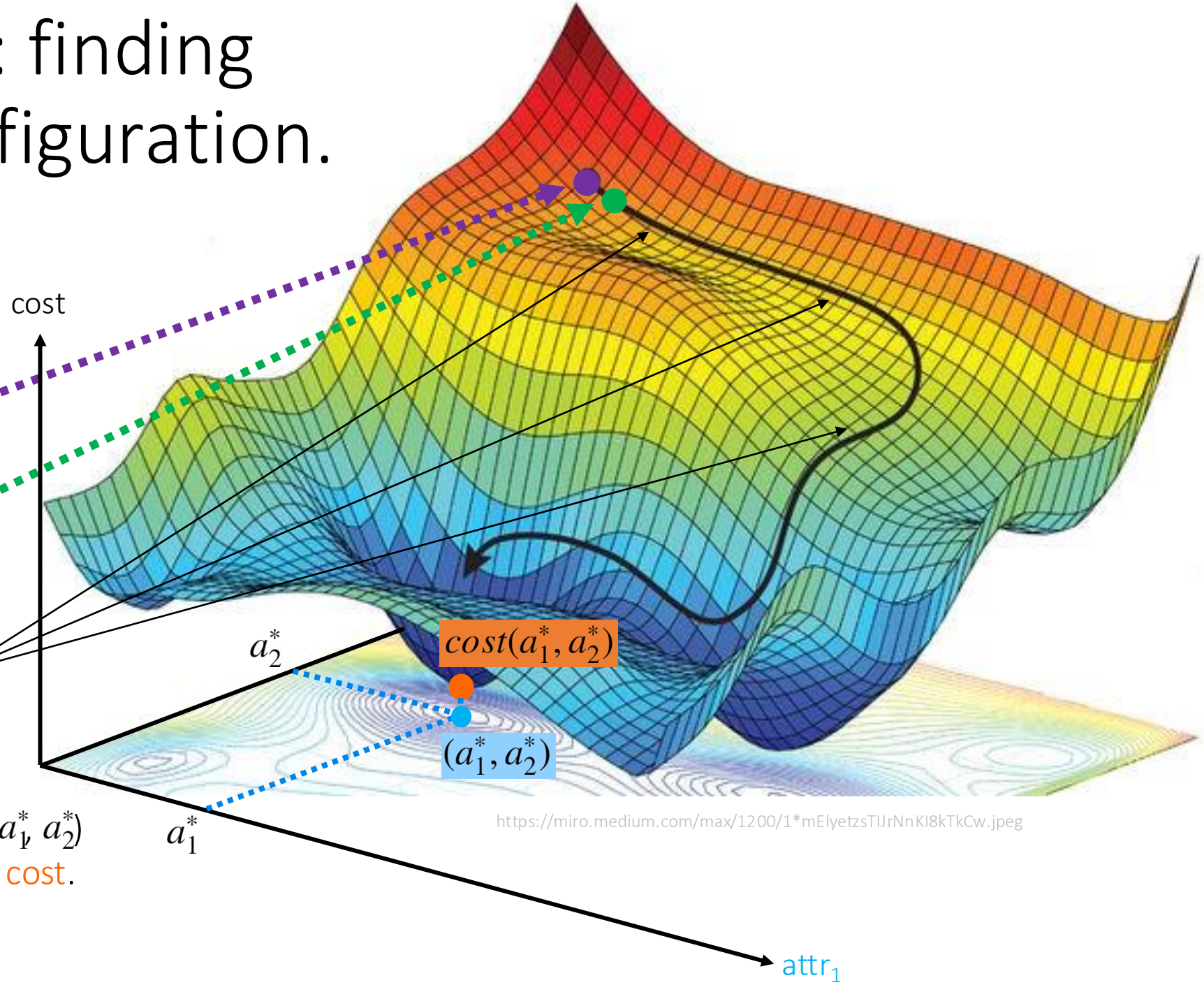
- Configuration: $attr_1$, $attr_2$.
- Each configuration has a resulting cost.
- Too expensive to just compute cost for all configurations.
- Goal: Find lowest-cost configuration.



Local search: finding the best configuration.

Suppose:

- The cost surface actually looks like this, but we don't know that.
- We just know current config and its cost.
- What config to try next? Try "close" ones, pick lowest resulting cost.
- Keeping changing the config to gradually move along the cost surface.

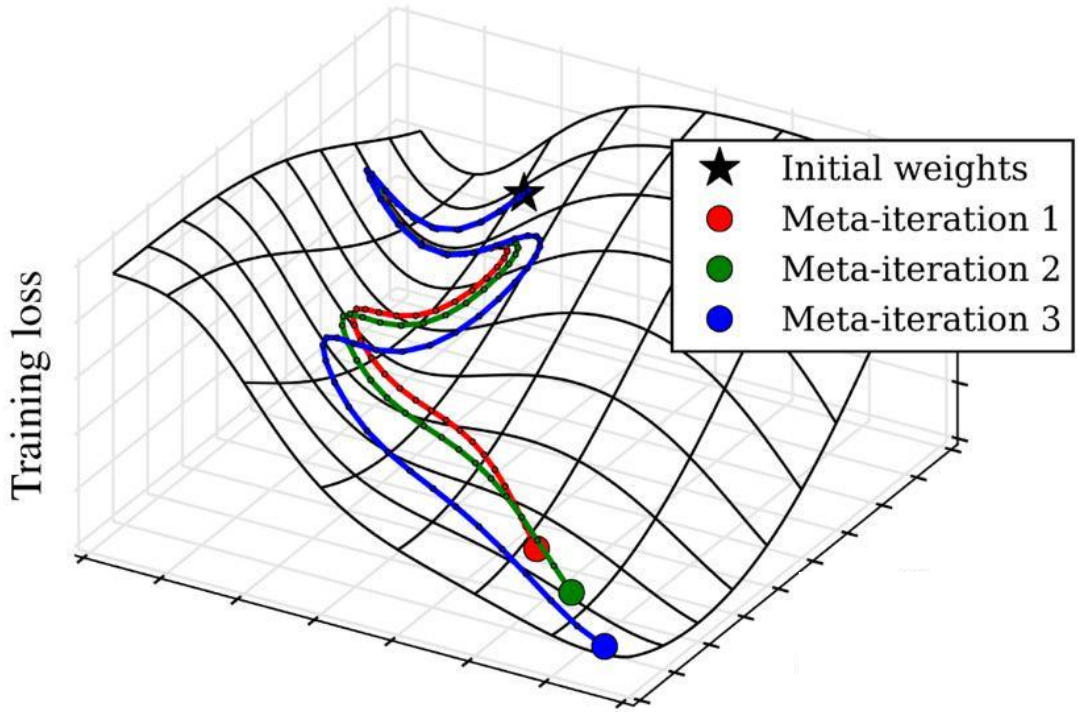


https://miro.medium.com/max/1200/1*mElyetzsTIJrNnKI8kTkCw.jpeg

- Find optimal configuration (a_1^*, a_2^*) corresponding to the lowest cost.

Example: Hyperparameter Search

- Many machine learning algorithms have hyperparameters that need to be set
  - e.g., What is the best learning rate?
- Use local search to find the best hyperparameters!
- Local search is behind all modern machine learning methods, all deep learning



https://3.bp.blogspot.com/-kaq-LFU3tQg/ViFM19wYkuI/AAAAAAAAQ0o/6QEHauXfw-U/s1600/weightiterations.jpg

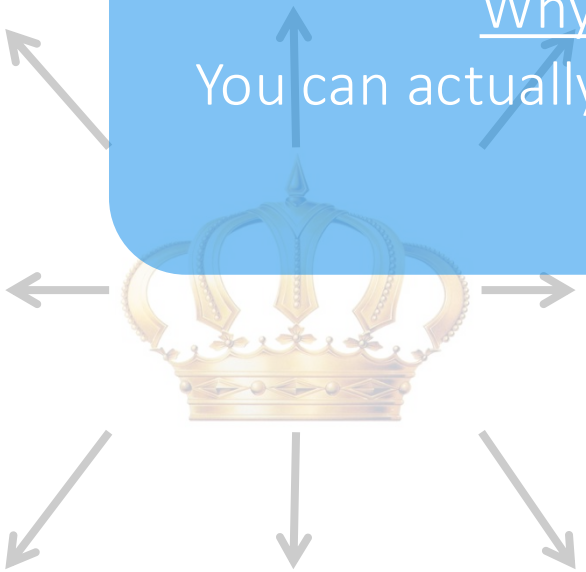
Other Examples of Local Search

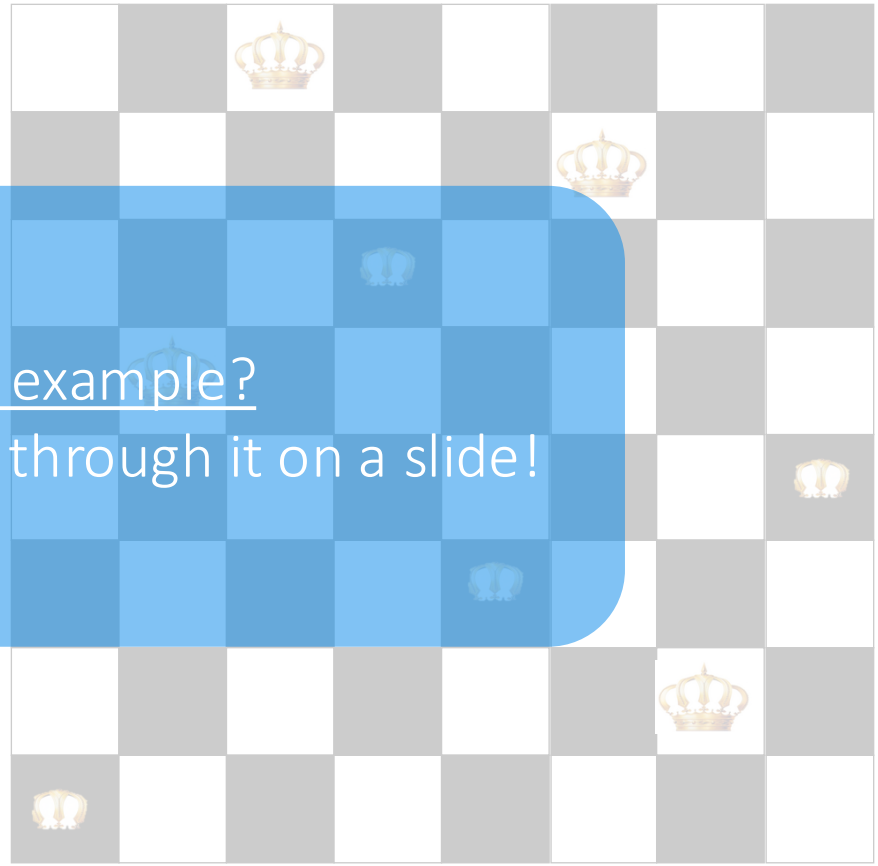
- Integrated circuit design
- Factory-floor layout
- Job-shop scheduling
- Telecommunications network optimization
- Vehicle routing

- In all cases – the final product is what matters, not the path!

Toy Problem: 8-Queens Problem

Place 8-queens such that no queen can attack any other queen



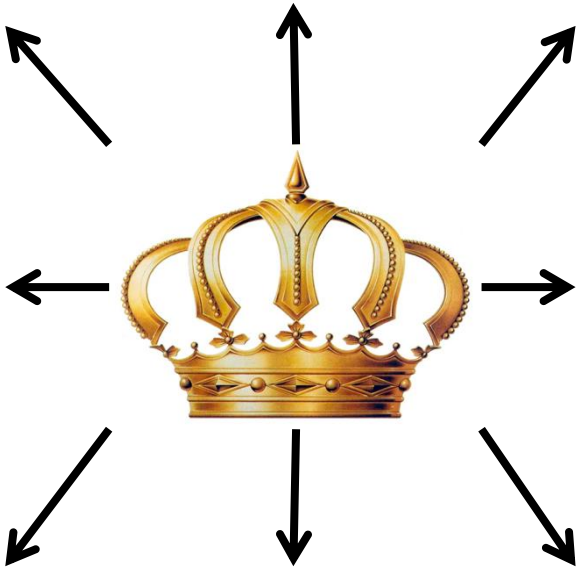


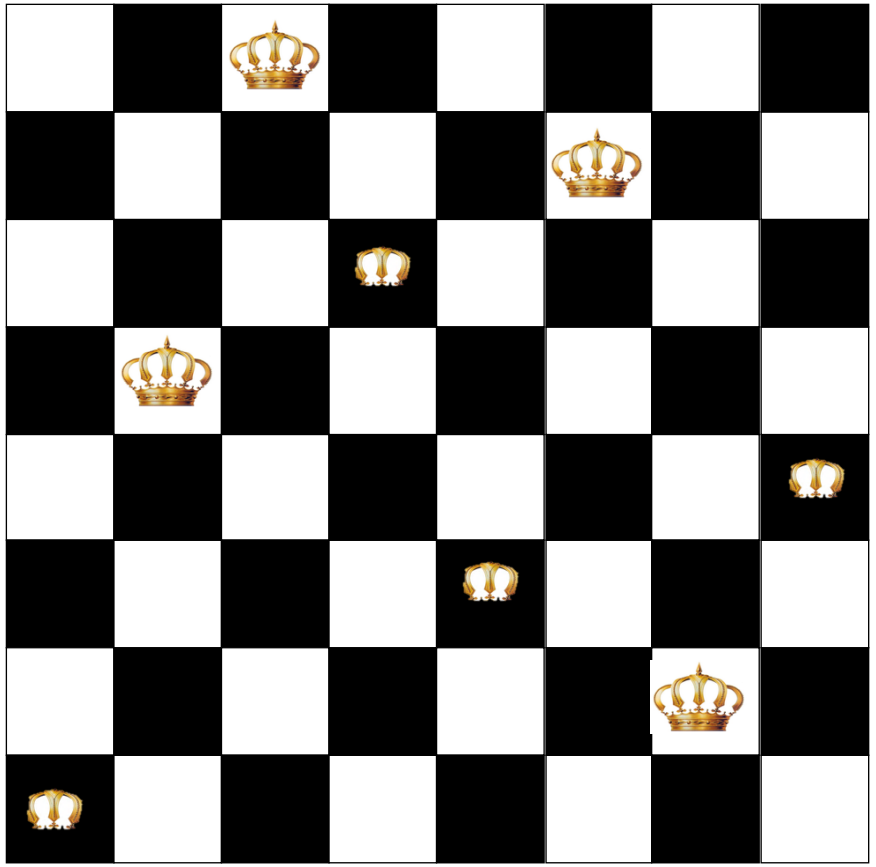
Why a toy example?

You can actually walk through it on a slide!

Toy Problem: 8-Queens Problem

Place 8-queens such that no queen can attack any other queen





When poll is active respond at PollEv.com/492

Formulate the Search Space

- How many placements of queens?
  - $64*63*62*\ldots*57 = 10^{14}$
- How many solutions?
  - Placement so queens won't be attacked: 2,057!
- How to start?
  - Iteratively add queens at every stage

Why is this a good candidate for local search?

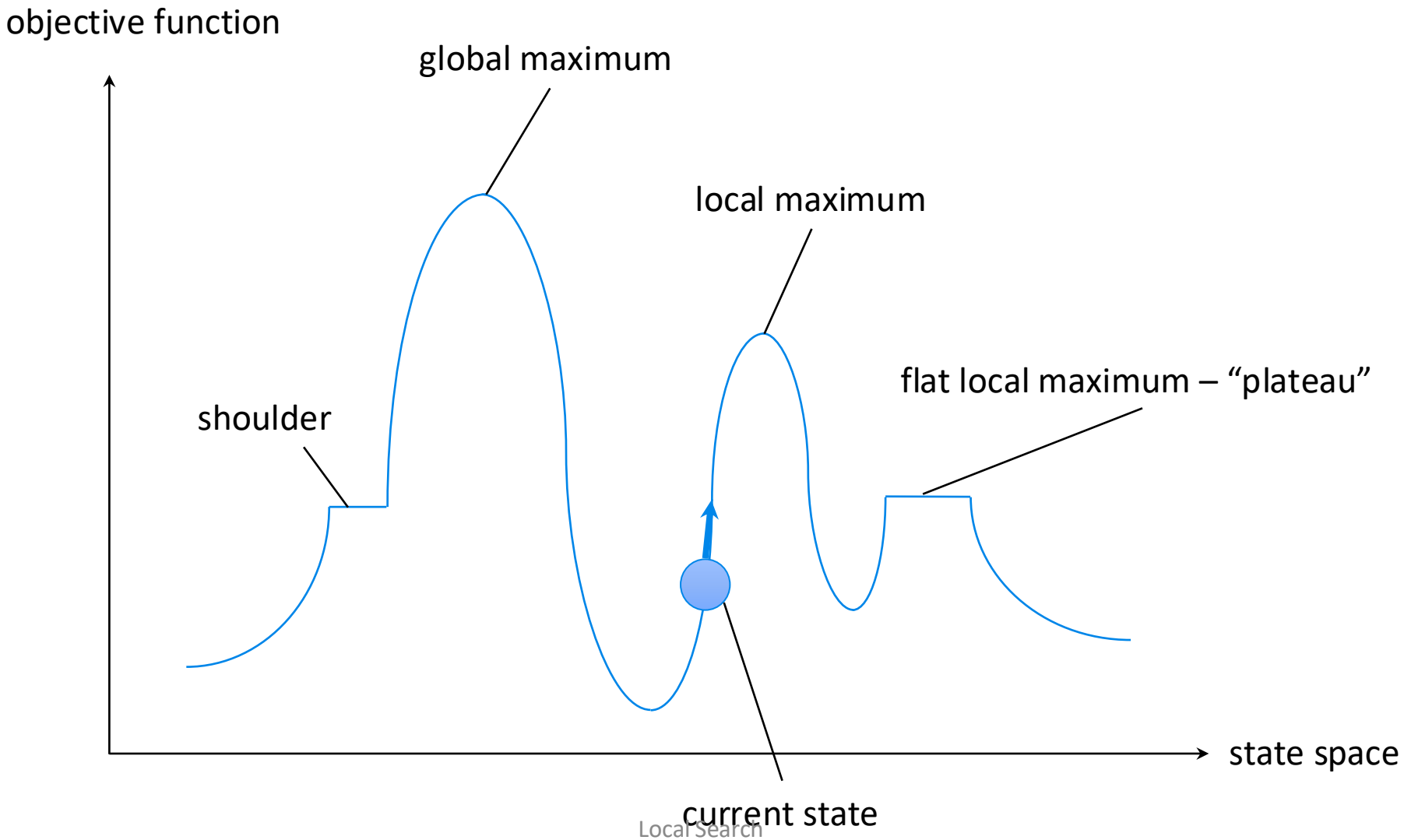
Local Search

- Basics:
  - Maintains only current node (not multiple paths)
  - Generally move only to neighbor
  - Paths are not retained
  - No real notion of path cost or goal test!

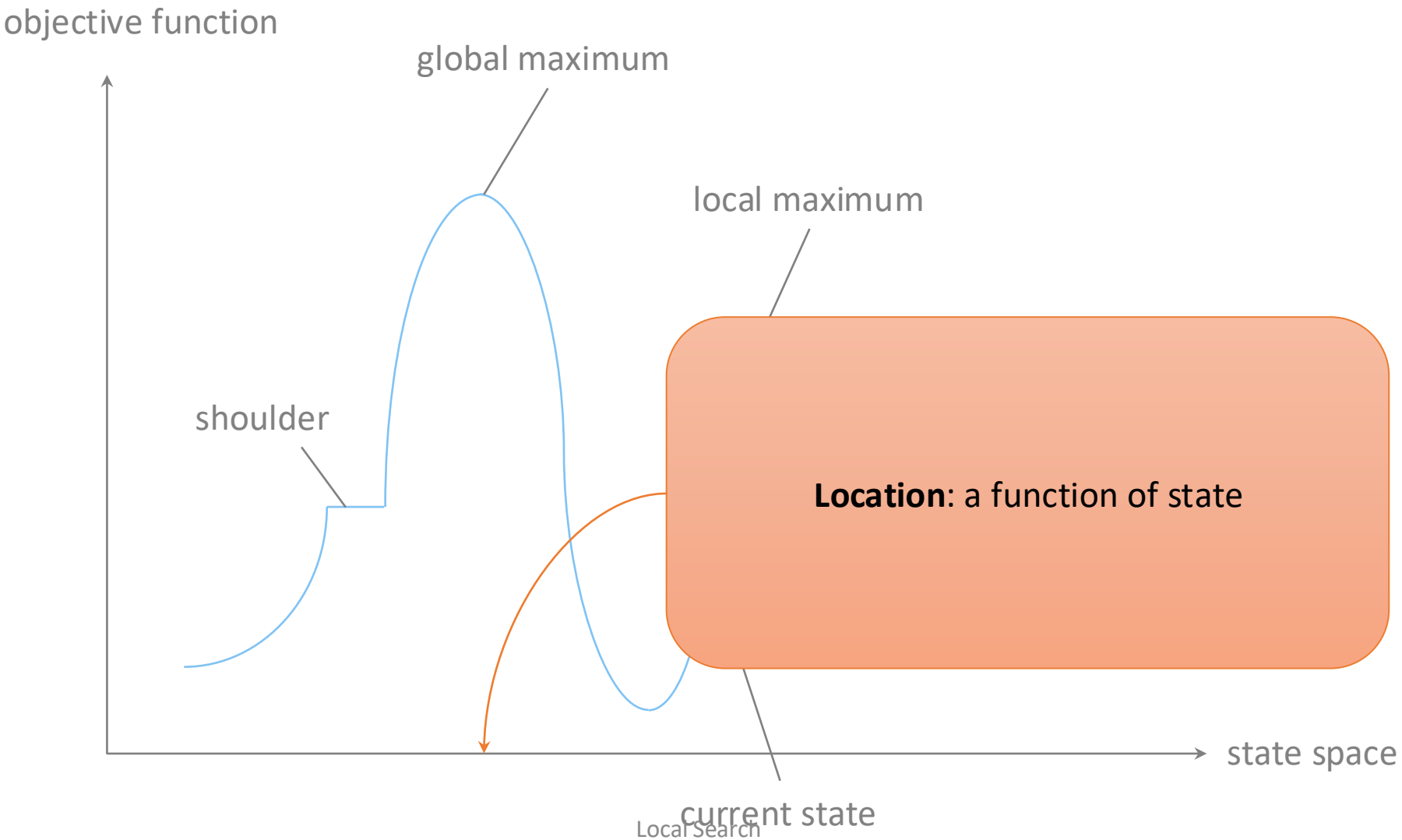
- Advantages:
  - Very low memory cost
  - Can often solve large continuous problems

- Can solve pure optimization problems:
  - Find the best state via an optimization function

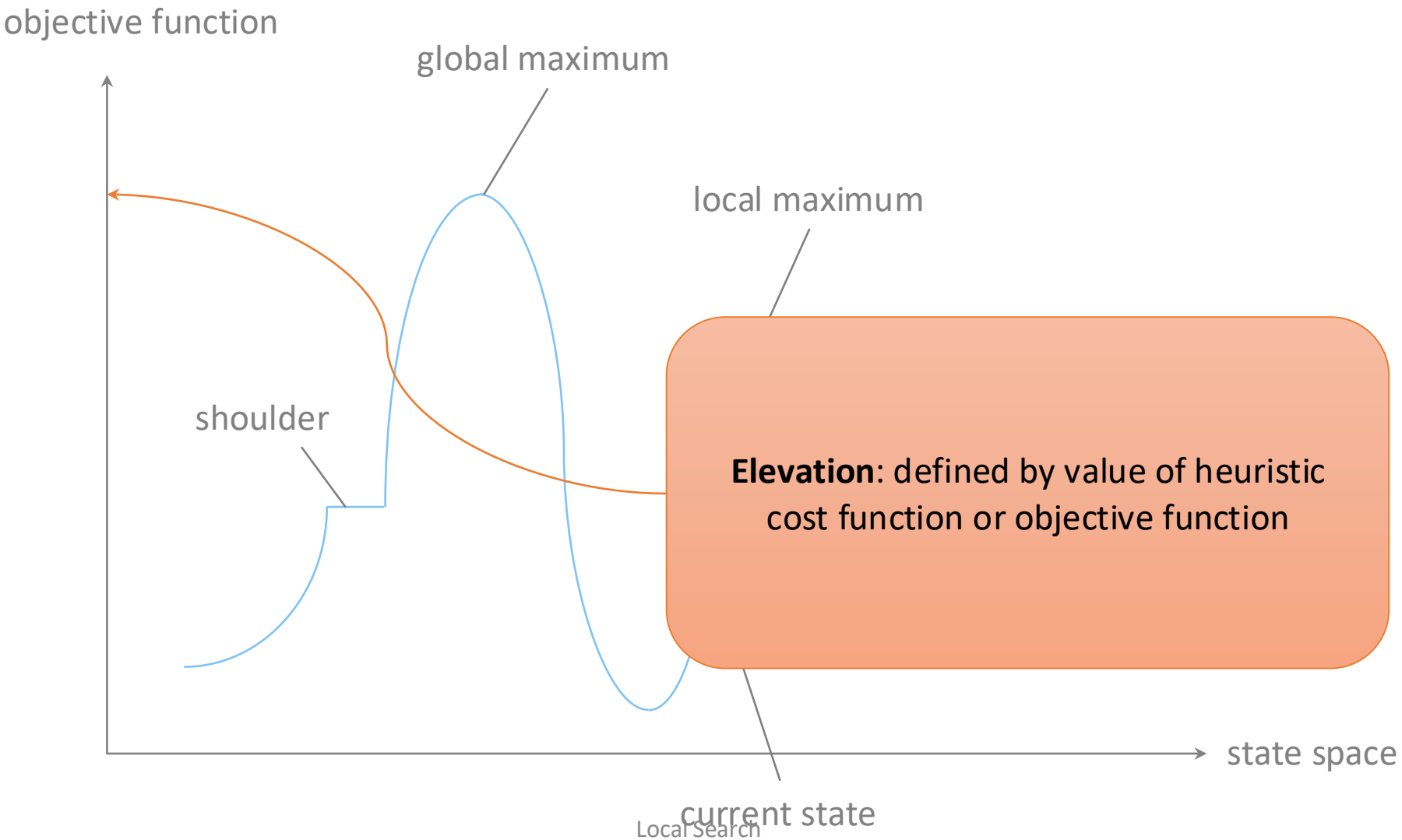
The State-Space



The State-Space

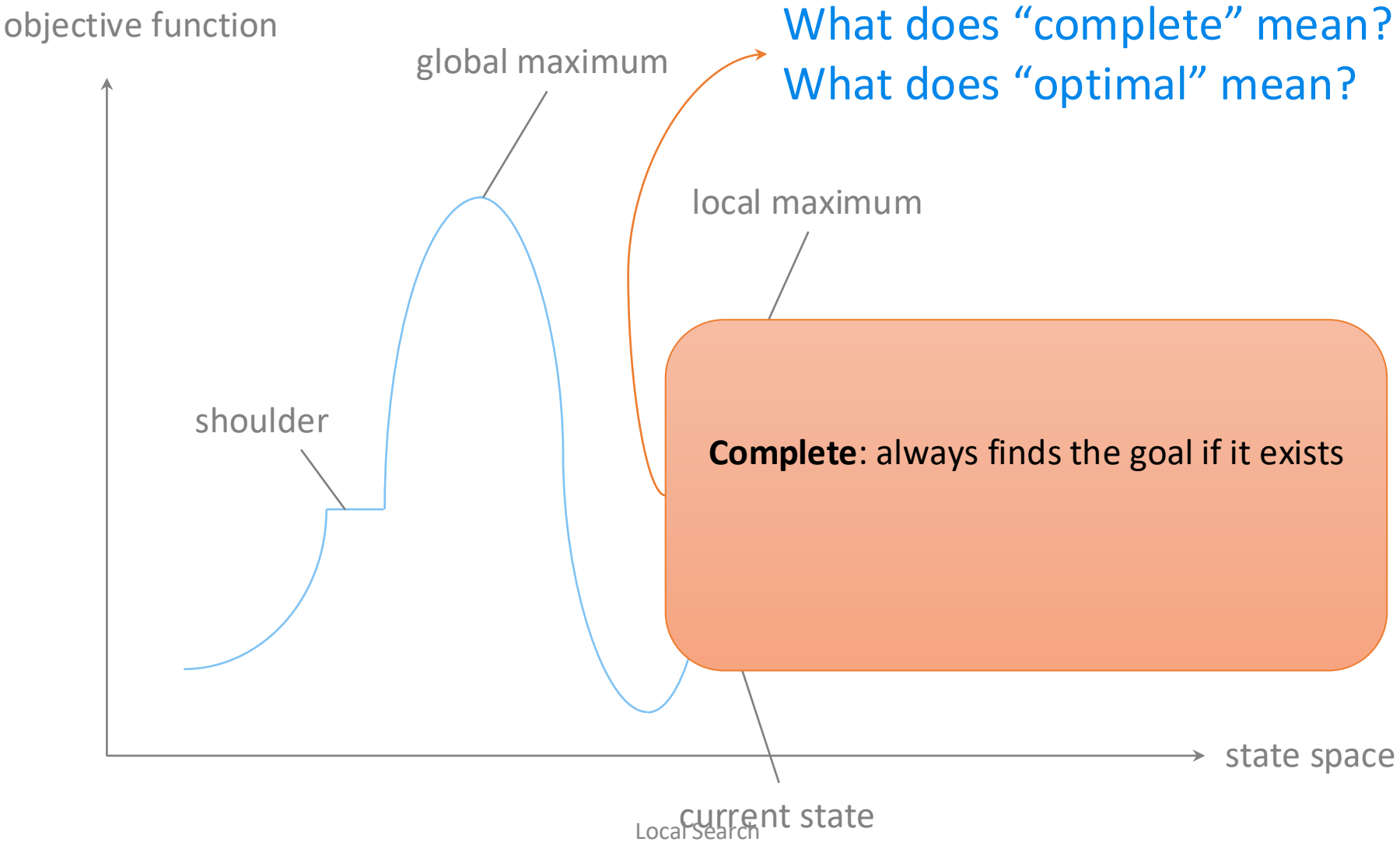


The State-Space



Elevation: defined by value of heuristic cost function or objective function

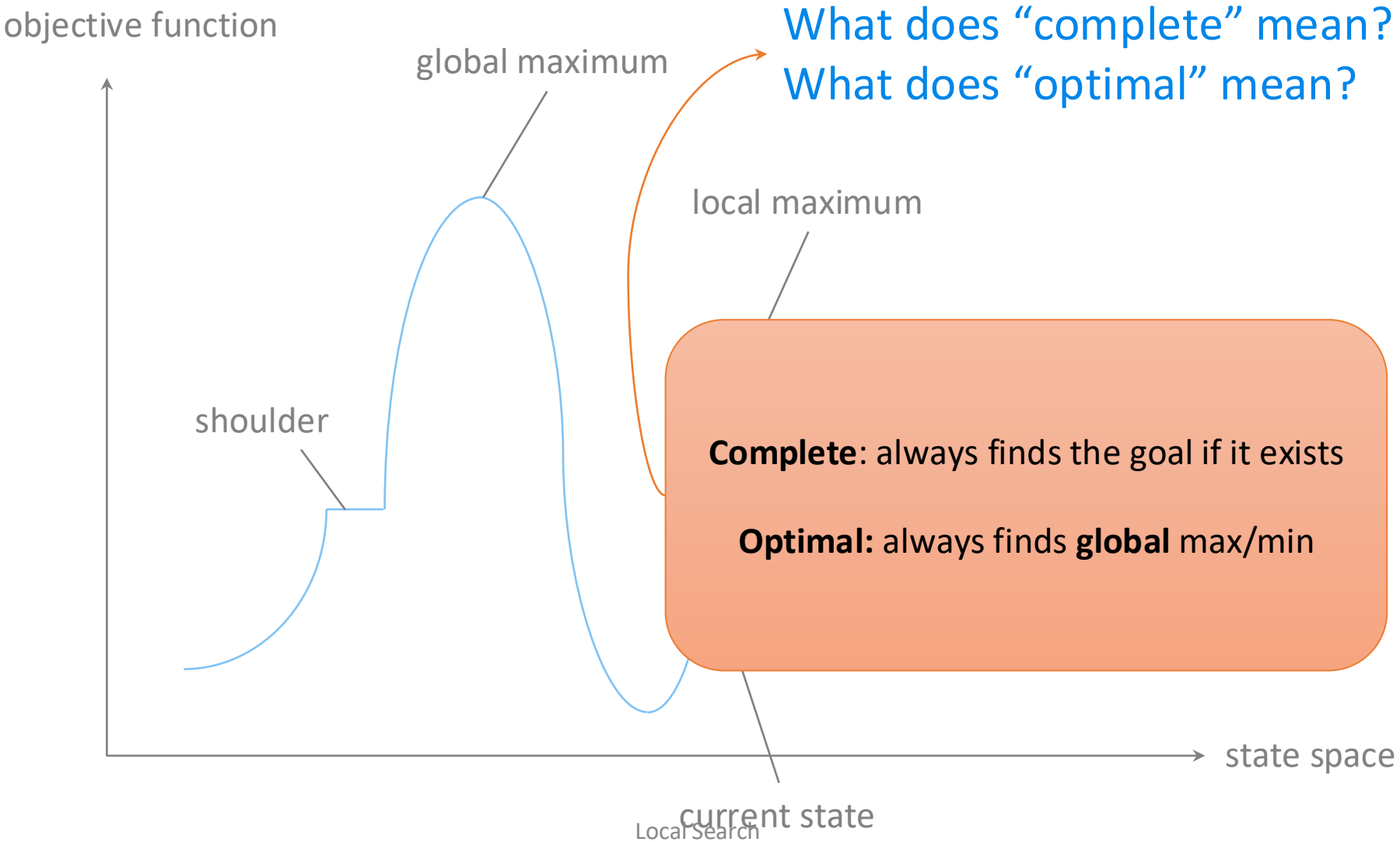
The State-Space



What does “complete” mean?
What does “optimal” mean?

Complete: always finds the goal if it exists

The State-Space



What does “complete” mean?
What does “optimal” mean?

Complete: always finds the goal if it exists

Optimal: always finds **global** max/min

Local Search Algorithms: Hill Climbing

Hill Climbing

- Overview:
  - Loop that continually moves in direction of increasing value
  - Terminates when it reaches a peak

- Important to note:
  - Does not maintain a search tree
  - Memory requirement: a single node!
  - Does not look beyond immediate neighbors

https://www.cs.cornell.edu/courses/cs4700/2011fa/lectures/04_LocalSearch.pdf

"This resembles trying to find the top of Mt. Everest in a thick fog while suffering from amnesia" (textbook)

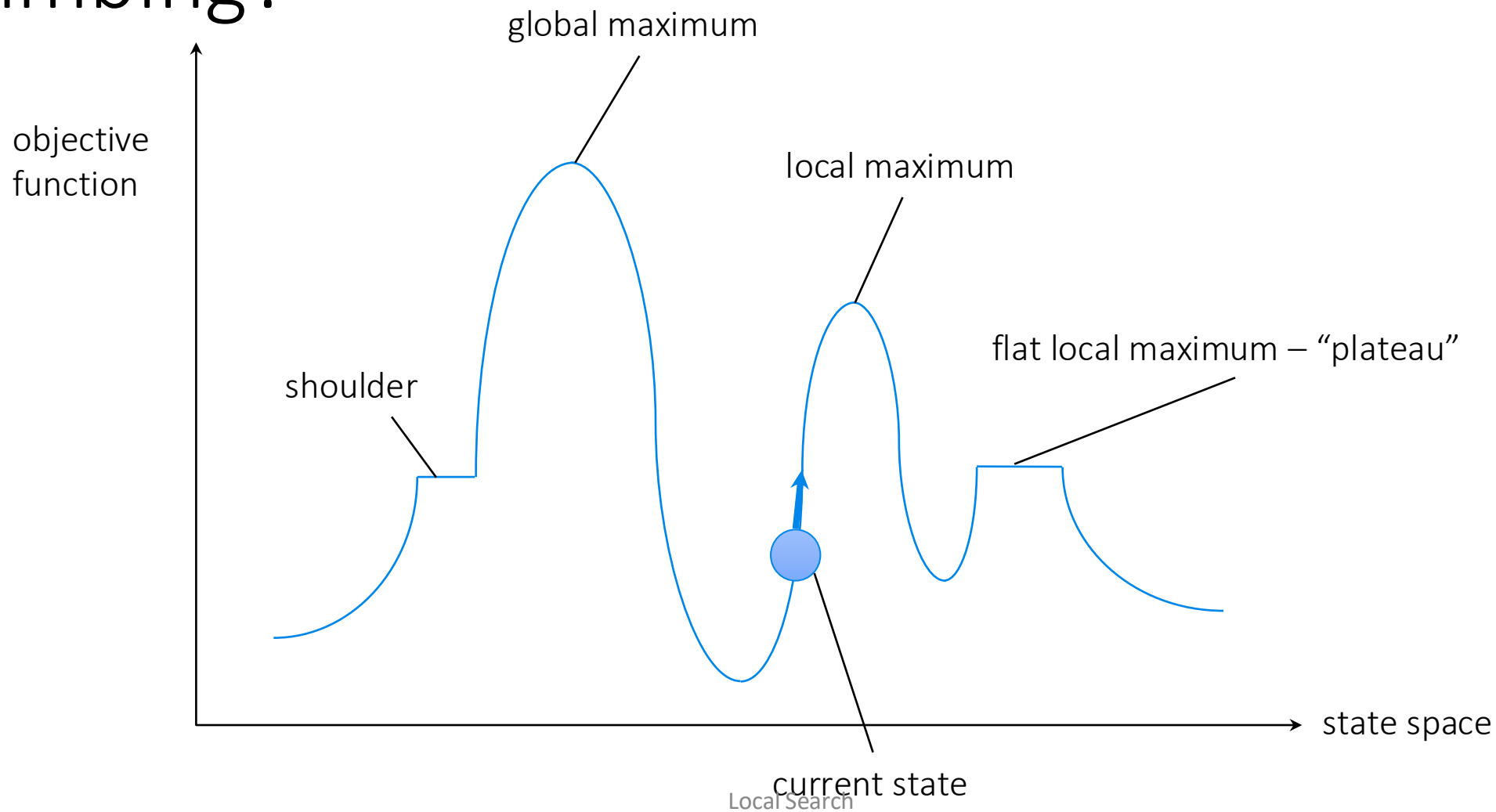
Hill Climbing

- Also called "Gradient Descent/Ascent"
- Requires "objective function" measuring the quality of the solution (could be a heuristic)

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← problem.INITIAL
    while true do
        neighbor ← a highest-valued successor of current
        if VALUE(neighbor) ≤ VALUE(current) then return current
        current ← neighbor
```

When poll is active respond at PollEv.com/492

What are some challenges with hill climbing?



Stochastic Variations

- Stochastic hill climbing
  - Select among positive steps at random
  - Probability proportional to steepness

Challenge: No clear answer on how often to restart from scratch versus trying to "repair" a current candidate that's stuck or making slow progress

- First-choice hill climbing
  - Generate successors randomly until you find a better one

- Random repeats
  - Repeat hill climbing from randomly chosen initial state
  - Return best local maximum found

Local Search Algorithms: Simulated Annealing

Annealing

- Process used to soften steel so that it can be easier to form
- Heat it up to a really high temperature (e.g., 100 degrees F)
- Then, gradually cool it off

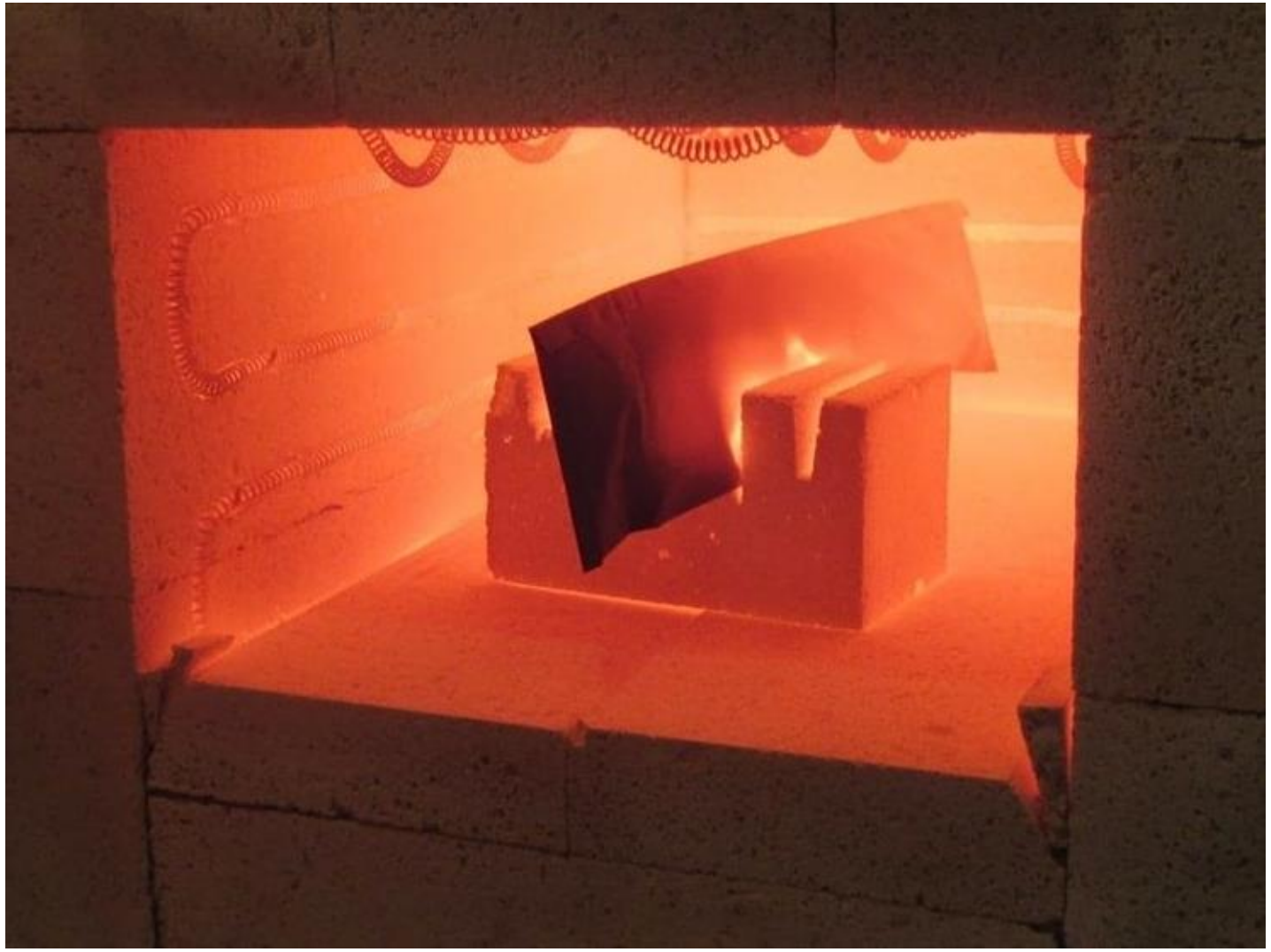
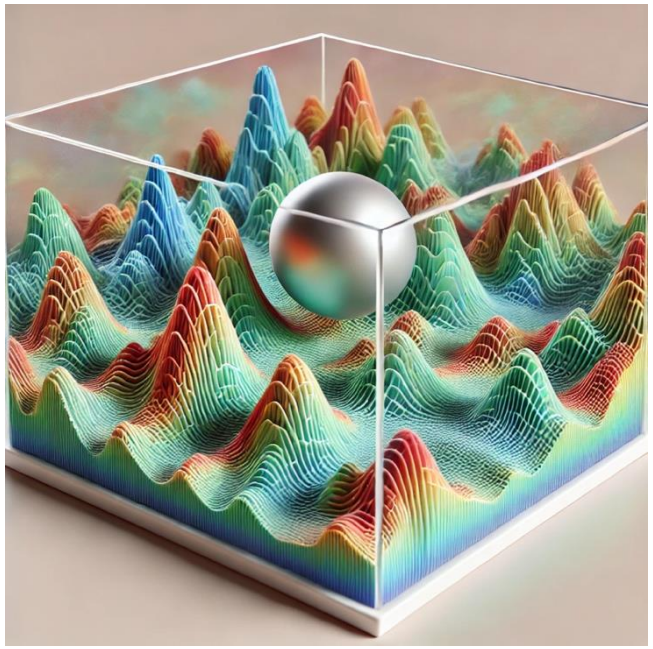


Image: https://makeitfrommetal.com/beginners-guide-on-how-to-anneal-steel/

Simulated Annealing: Intuition



- Picture a ball in a box rolling down a bumpy surface…

- Simulated Annealing (steepest descent analogy)
  - At first – shake the box hard
  - Gradually reduce the intensity of the shaking
  - High temperature → low temperature
    (worse moves often chosen) → (worse moves rarely chosen)

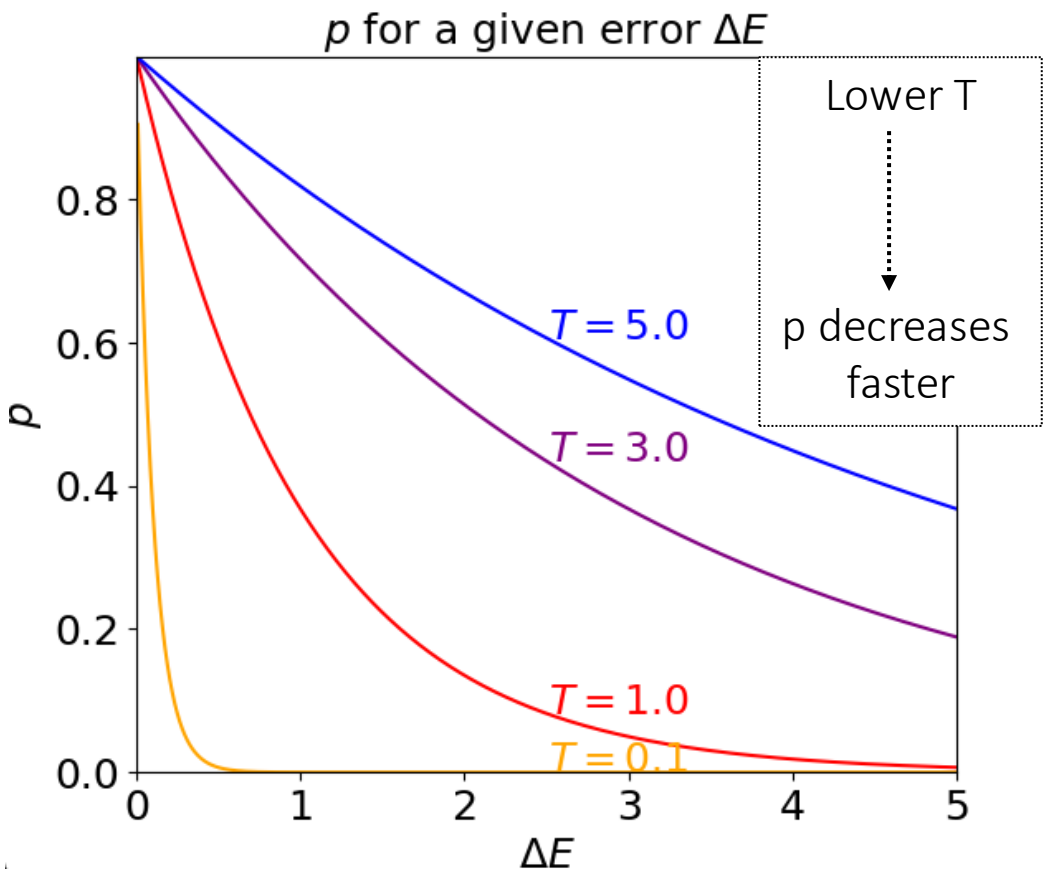
- Similar idea applies to steepest ascent

```
function SIMULATED-ANNEALING(problem, schedule) returns a state
  current ← problem.INITIAL
  for t = 1 to ∞ do
    T ← schedule(t)
    if T = 0  then return current
    next ← a randomly selected successor of current
    ΔE ← VALUE(current) - VALUE(next)
    if ΔE ≤ 0 then current ← next
    else current ← next only with probability p = e^(-ΔE/T)
```

- In this version, we're trying to ***maximize*** `value(state)`.
  - **WARNING**: Book pseudocode has problems…
- T controls how willing we are to try a bad (downhill) move.
  - Starts high (willing), gradually gets low (less willing)
  - T = 0, stop
- ΔE is how bad `next` is.
  - Higher number means even worse
  - Negative number means `next` is actually better

Example:

- value(current)=5
- value(next)=6 (better!)
- ΔE = value(current) – value(next) = -1.
- ΔE ≤ 0, so we accept next and iterate.

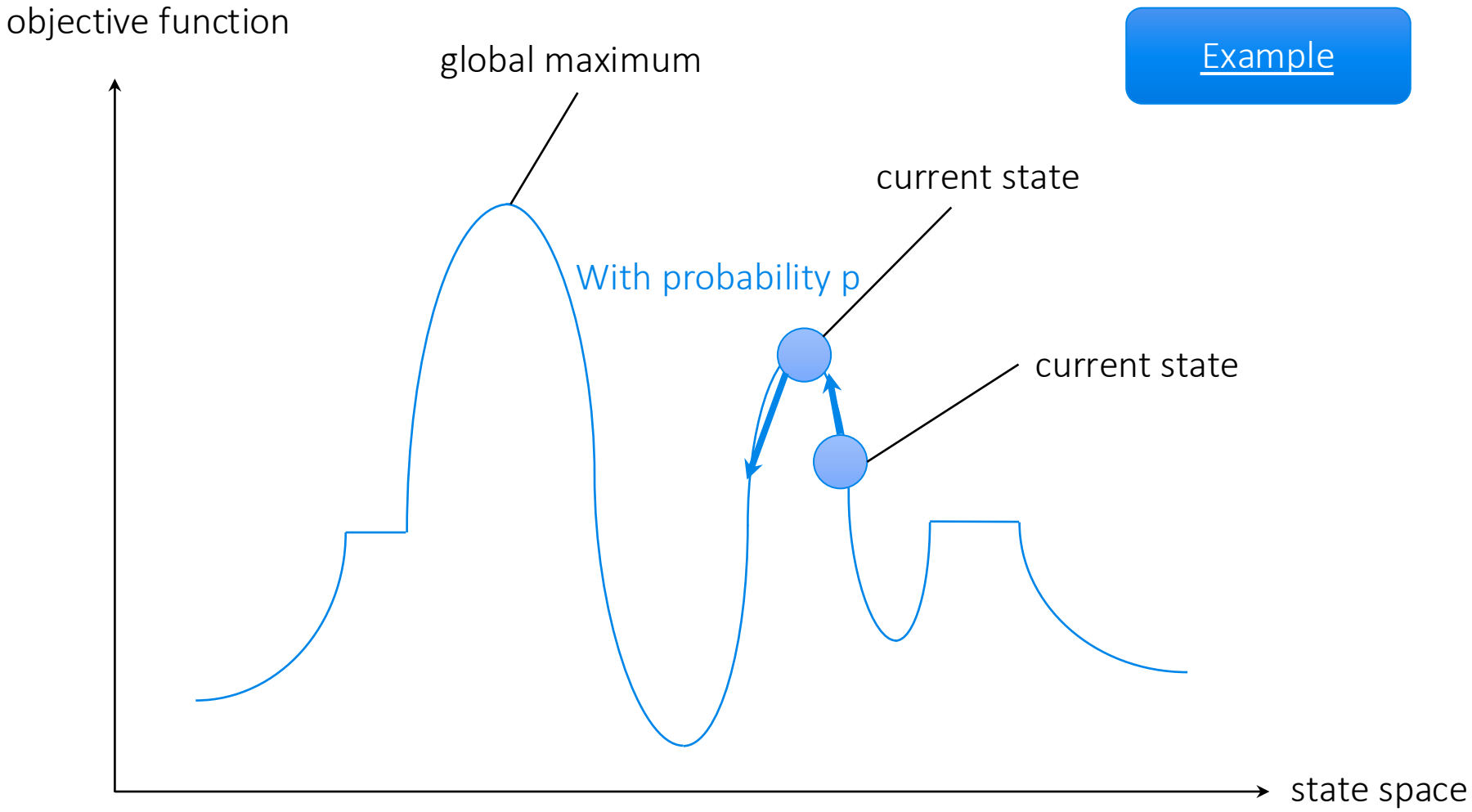


Larger ΔE → lower p

- New state is a little worse than old state
- Small positive ΔE
- p is higher

- New state is much worse than old state
- Larger ΔE
- p is lower

Simulated Annealing: Example



Example

New Slide

Exploration vs. Exploitation

- Common theme in AI
- In simulated annealing:
  - High T means preference for exploration
    - More likely to make "bad" moves to explore more of the search space
      even though we might leave a good area.
  - Low T means preference for exploitation
    - Less likely to make "bad" moves
      even though this may mean staying in a suboptimal area.
  - Plan: Explore at first (high T), gradually settle on exploitation (low T).

Updated

Simulated Annealing

- Possible schedules:
  - Linear cooling: update with $T(t) = T_0 - \alpha t$
  - Geometric reduction: update with $T(t) = T_0 * \alpha^t, \quad 0 < \alpha < 1$
  - Slow decrease rule: update with $T(t) = \frac{T_0}{1+\beta t}$

    β is adjusted based on whether recent moves have been accepted or rejected
  - And others …

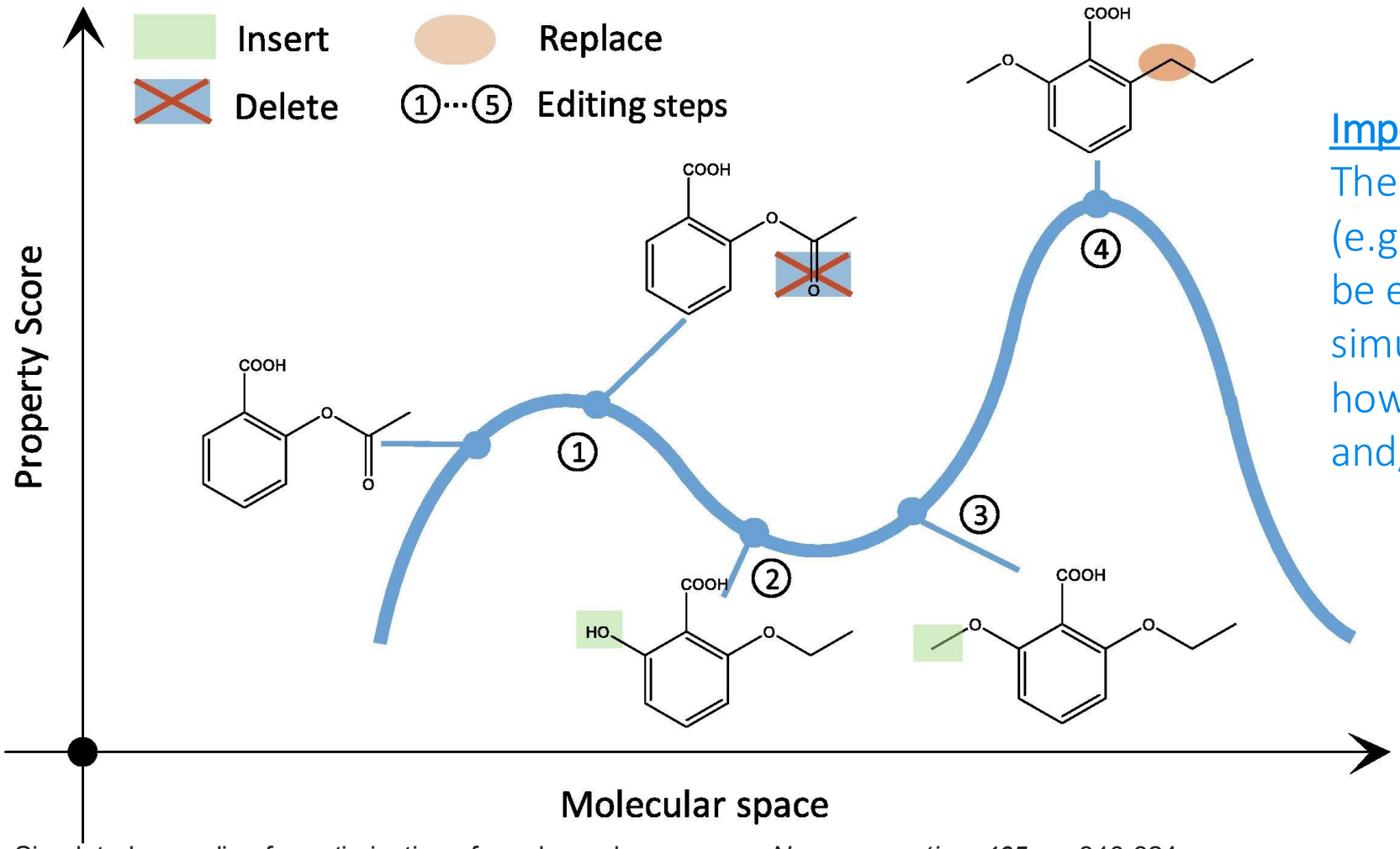
T_0: initial temperature

- Important:
  - If T is scheduled properly for a given domain and the process continues until T=0, simulated annealing has high probability of finding a near-optimal solution.

Gelfand, Saul B., and Sanjoy K. Mitter. "Analysis of simulated annealing for optimization." *1985 24th IEEE Conference on Decision and Control*. IEEE, 1985.

New Slide

Application to molecule structure optimization



Important:
The evaluation function (e.g., property score) can be expensive (e.g., simulation); this will limit how many time steps and/or restarts we can do

Liu, X., et al., 2021. Simulated annealing for optimization of graphs and sequences. *Neurocomputing*, *465*, pp.310-324.

Local Search Algorithms: Local Beam Search

So far: we are only keeping track of one node in memory!

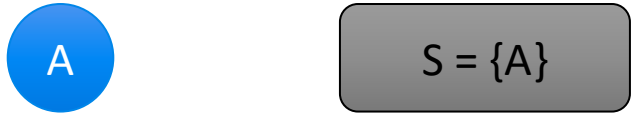
Local Beam Search

- New addition: keep track of k states (k > 1)

- Process:
  1. Randomly generate k initial states
  2. Generate successors for each state
  3. If any successor is a goal, return it and exit
  4. Otherwise put all successors into queue and sort
  5. Remove all but the k best nodes from queue, go to step 2

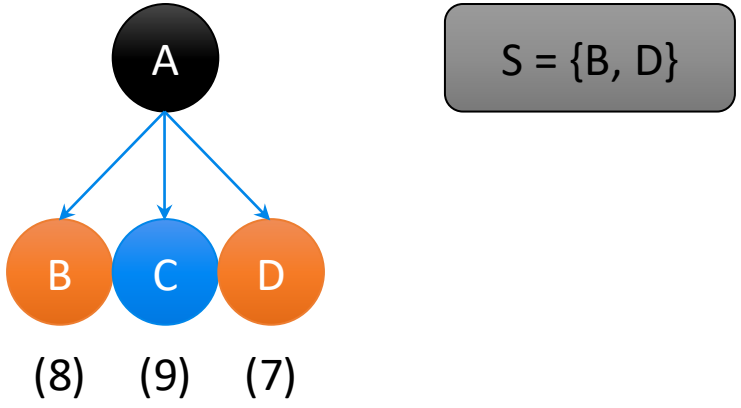
Local Beam Search – Example

- Let’s assume k = 2



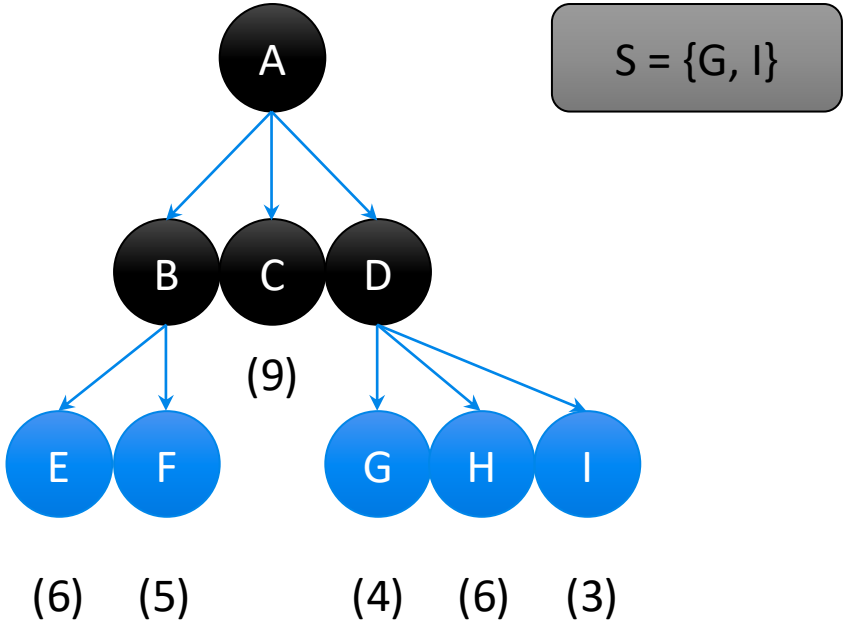
Local Beam Search – Example

- Let's assume k = 2



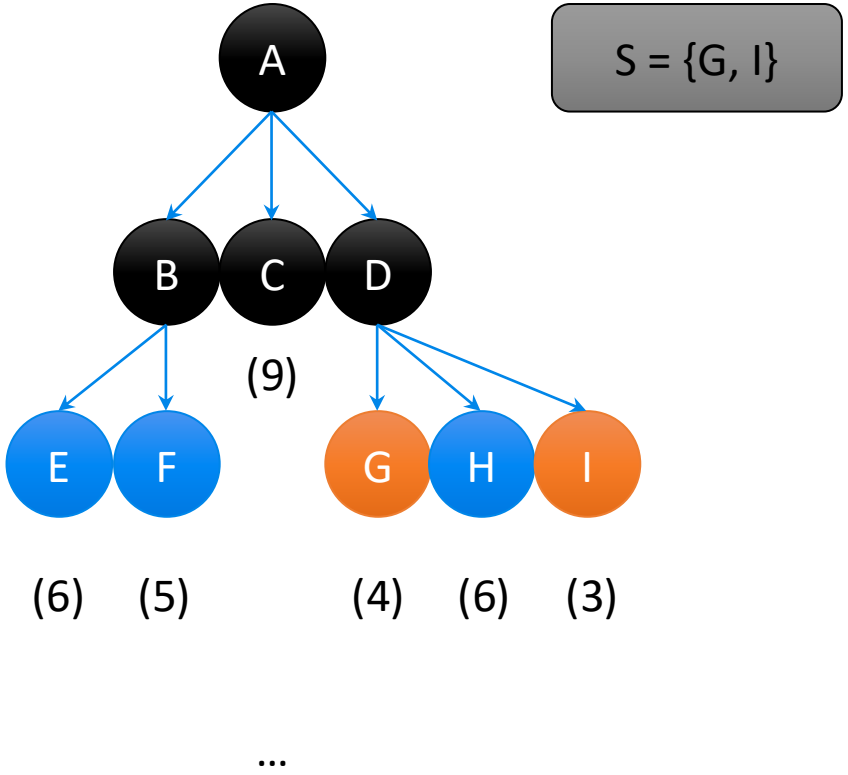
Local Beam Search – Example

- Let's assume k = 2



Local Beam Search – Example

- Let's assume k = 2



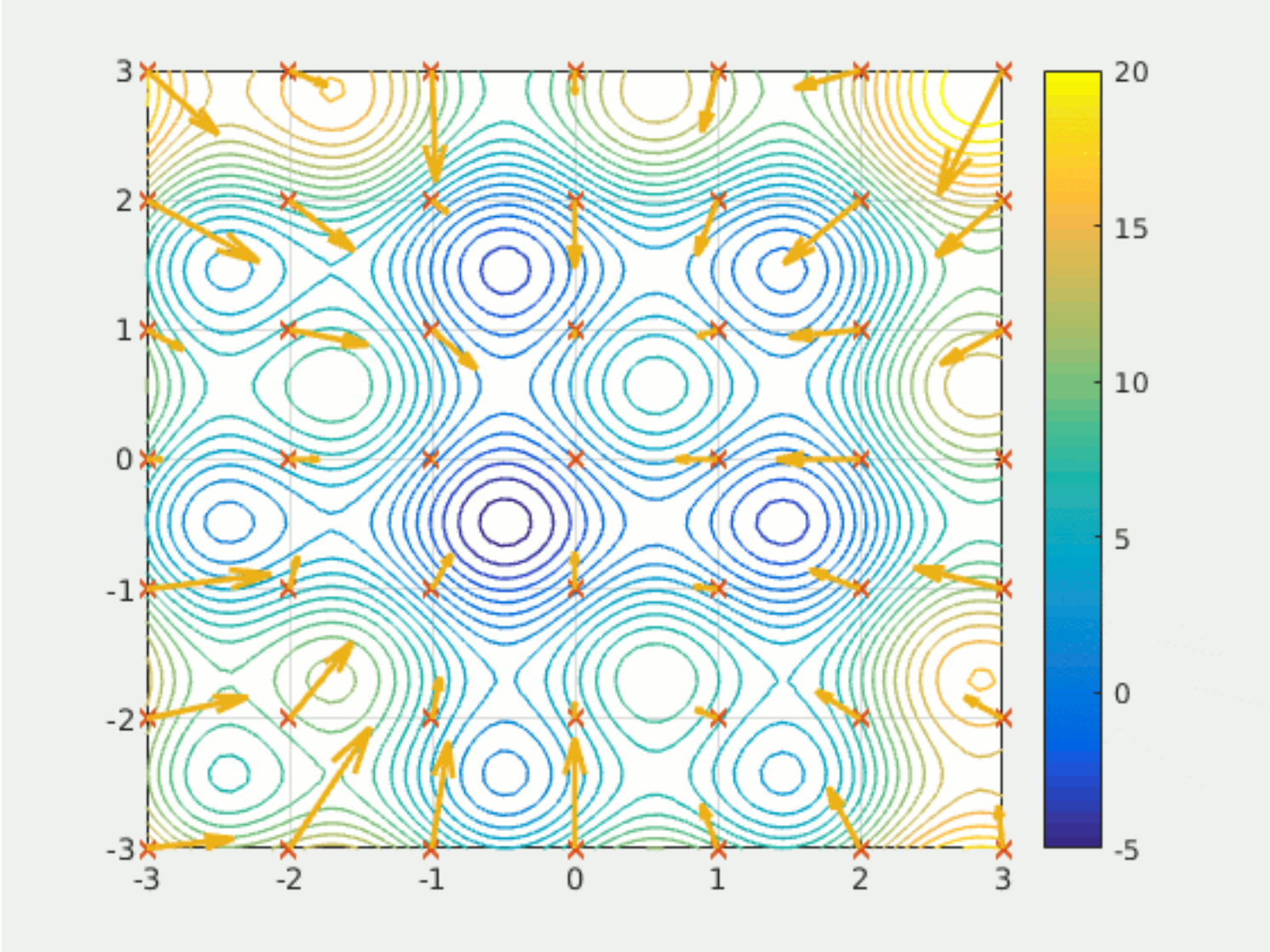
Local Beam Search

- Is there an advantage over running k random restarts in parallel?
  - Yes!
  - Random restart – searches independently
  - Local beam search – useful information passed along parallel threads
- Benefit: algorithm abandons unfruitful searches
  - Moves resources to where progress is made!
- What if diversity of states is low?

Stochastic Beam Search

- Choose k successors
  - Not looking for the very best
  - Select proportional to heuristic value

Extra discussion: Swarm Intelligence



Local Search Algorithms: Genetic Algorithms

Inspiration: Theory of Evolution

Can be thought of as a variant of stochastic beam search – combining successors from different parents

- What if we use more than the current state…
- What if we use properties from multiple states?

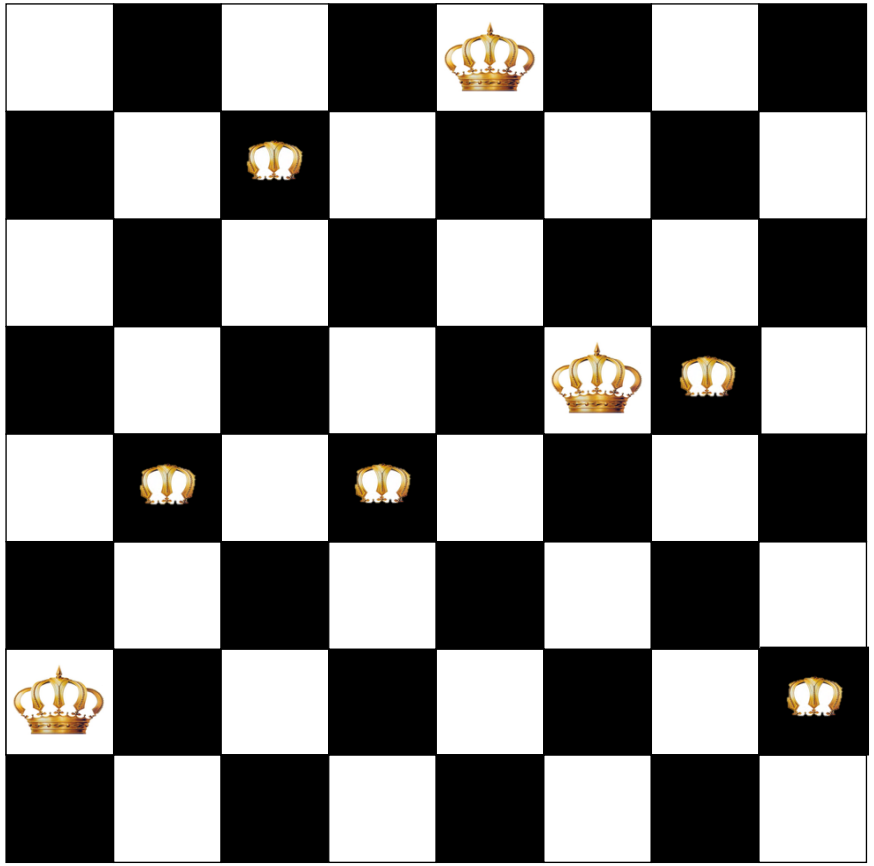
+

=

Updated Slide

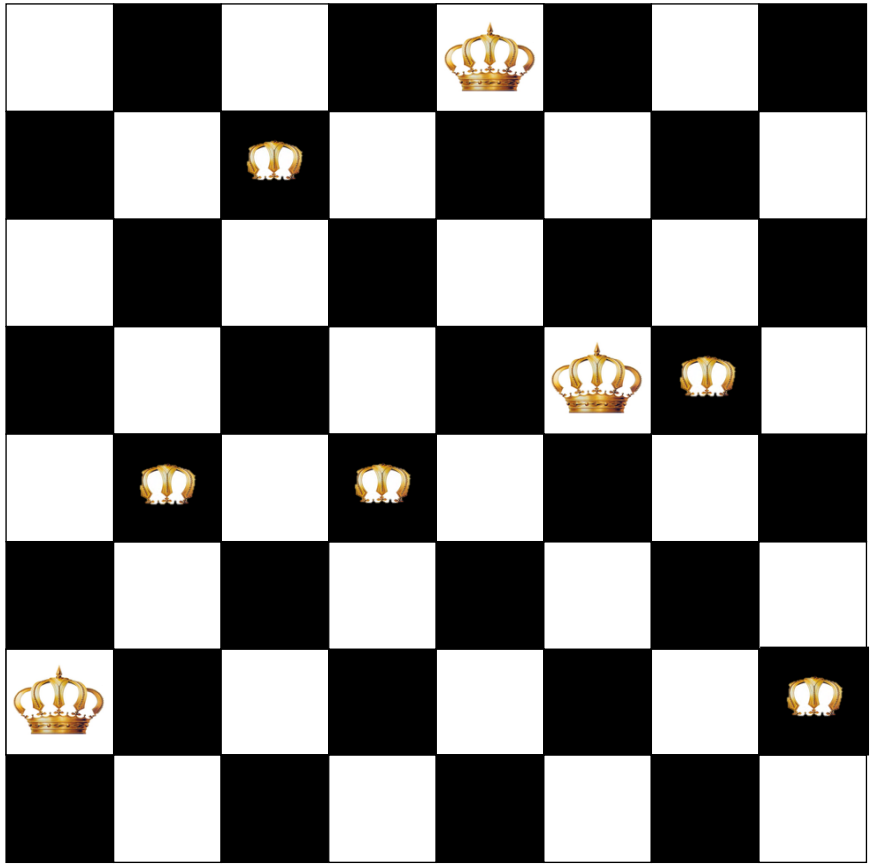
Back to the 8-Queens Problem

- Let's describe the state of the board...
  - How about the row position of each column?
- Fitness function:
  - Number of *nonattacking* pairs of queens



New Slide

Specifying the State



Description of Current State:

2	4	7	4	8	5	5	2

New Slide

Fitness Function

Fitness function to **maximize**:
For each Q,
count the number of Q's to its right
that are ***not*** in conflict.

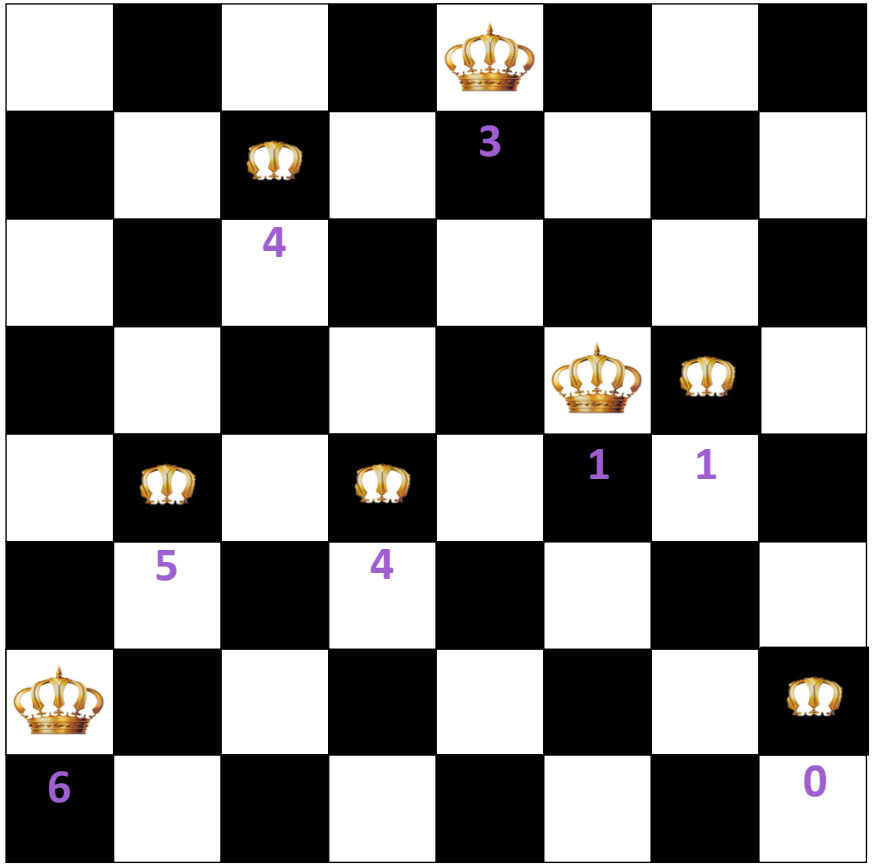
fitness = sum of these

This example:

6 + 5 + 4 + 4 + 3 + 1 + 1 + 0 = 24

Ideal:

7 + 6 + 5 + 4 + 3 + 2 + 1 + 0 = 28



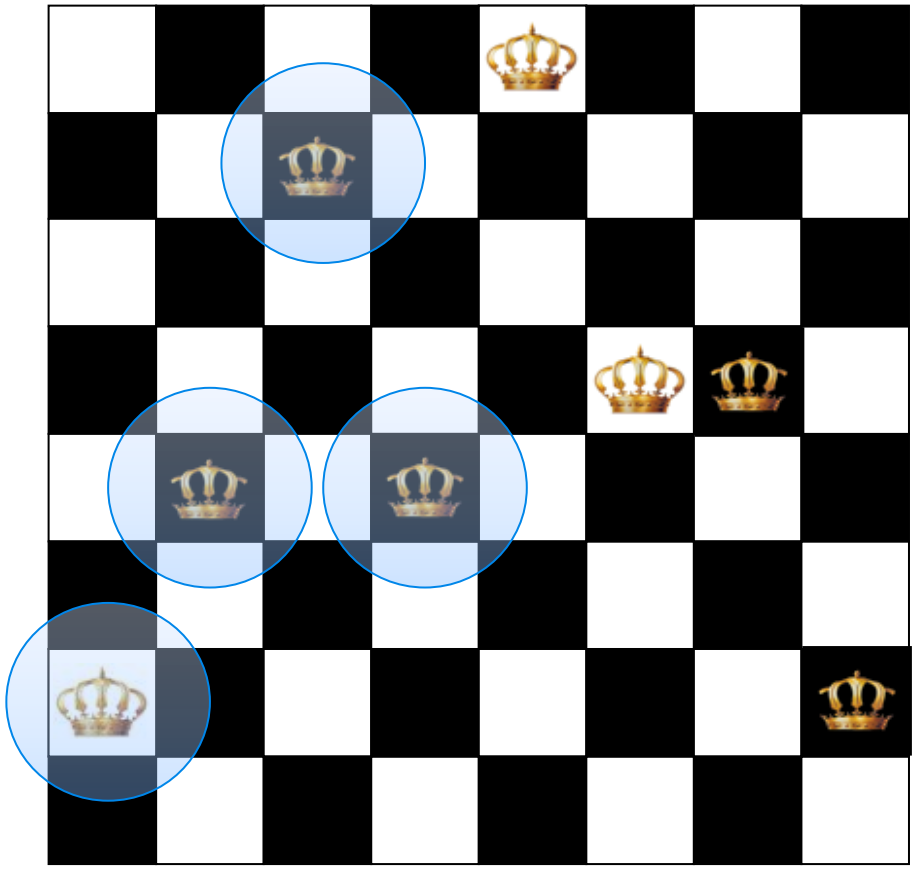
2	4	7	4	8	5	5	2

Genetic Algorithms – Example

Initial Configurations:

2	4	7	4	8	5	5	2
3	2	7	5	2	4	1	1
2	4	4	1	5	1	2	4
3	2	5	4	3	2	1	3

Board State for First Configuration:



Fitness Function (ideal in gray):

6	5	4	4	3	1	1	--

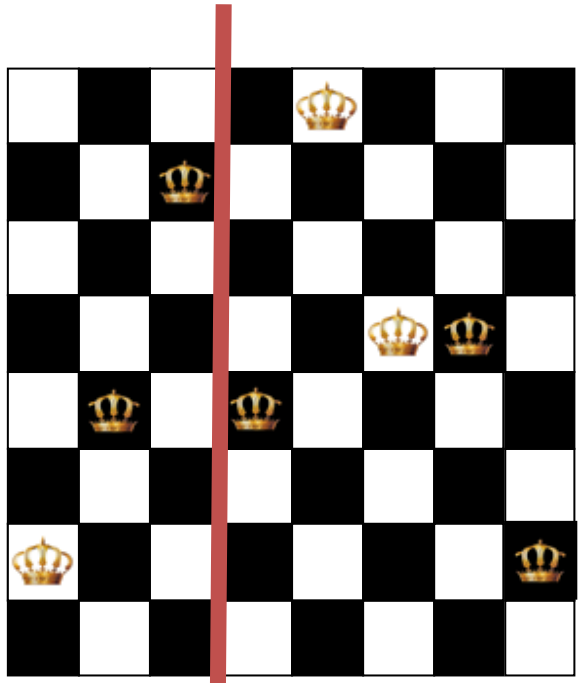
→ 24

7	6	5	4	3	2	1	--

Crossover and Mutation

Pictorial Example

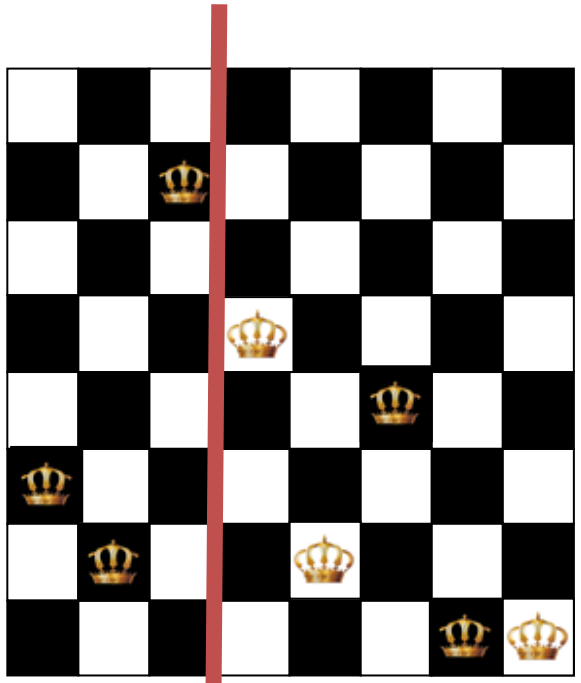
Parent 1:



2	4	7	4	8	5	5	2

+

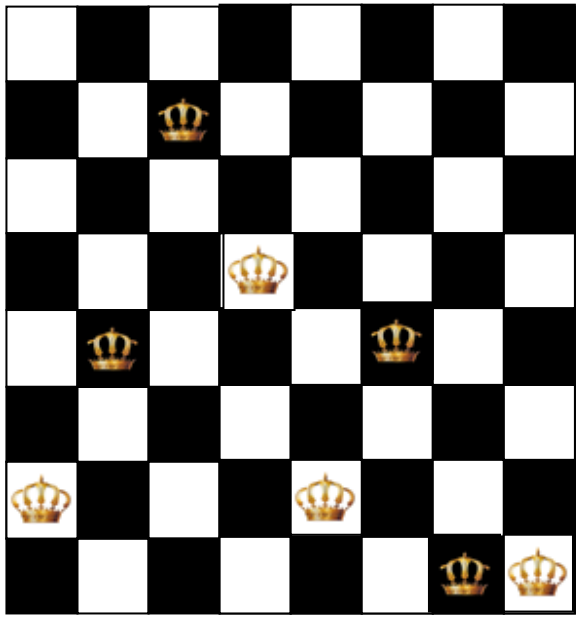
Parent 2:



3	2	7	5	2	4	1	1

=

Child:



2	4	7	5	2	4	1	1

New Slide

Genetic Algorithms – Overview

```
function GENETIC-ALGORITHM(problem, fitness) returns an individual
    repeat
        weights ← WEIGHTED-BY(population, fitness)
        population2 ← empty set
        for i = 1 to SIZE(population) do
            parent1, parent2 ← WEIGHTED-RANDOM-CHOICES(population, weights, 2)
            child ← REPRODUCE(parent1, parent2)
            if (small random probability) then child ← MUTATE(child)
            add child to population2
        population ← population2
    until some individual is fit enough, or enough time has elapsed
    return the best individual in population, according to fitness
```

```
function REPRODUCE(parent1, parent2) returns an individual
    n ← LENGTH(parent1)
    c ← random number from 1 to n
    return Append(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c+1, n))
```

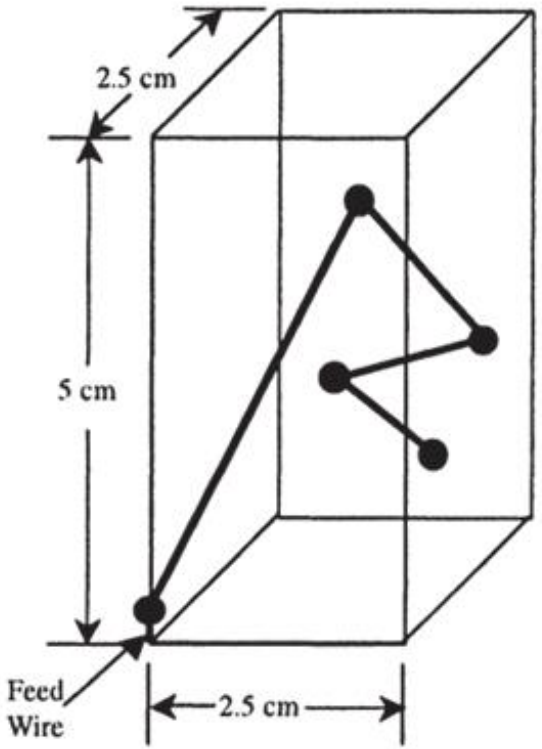
Genetic Algorithms – Discussion

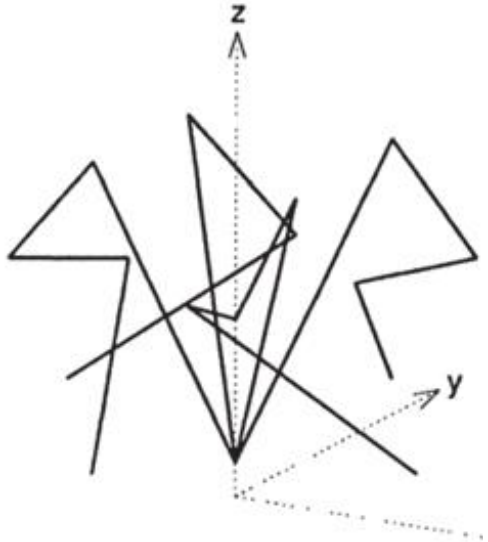
- Appealing analogy to natural selection
- Exchange of information across parallel search threads!

- Does it work?
  - Hard to characterize in general
  - Depends crucially on the string representation of state
  - Generally better than non-stochastic search methods
    But not generally better than simpler stochastic search methods

Example: NASA Antenna

- Evolved to meet mission requirements





- `forward(length, radius)` - add a wire with the given length and radius extending from the current location and then change the current state location to the end of the new wire.
- `rotate-x(angle)` - change the orientation by rotating it by the specified amount (in radians) about the x-axis.
- `rotate-y(angle)` - change the orientation by rotating it by the specified amount (in radians) about the y-axis.
- `rotate-z(angle)` - change the orientation by rotating it by the specified amount (in radians) about the z-axis.

Lohn, J. "An evolved antenna for deployment on NASA's space technology 5 mission." *Genetic Programming Theory and Practice II* (2005): 301-315.

Summary of Local Search

- Key advantages
  - Very little memory needed (O(1))
  - Can often find reasonable solutions in large or infinite (continuous) state spaces where other systematic approaches are unsuitable

- Key disadvantages
  - Usually incomplete
  - Usually not optimal